

Deep Learning-Based Lunar Crater Detection Using Modern Object Detection Architectures

Scott Nguyen

University of New Mexico ECE 533 Spring 2026

Abstract—This paper presents a comparative study of modern deep learning object detection architectures for automated lunar crater detection using annotated planetary surface imagery. YOLOv8 Nano, Faster R-CNN, and RetinaNet were implemented and evaluated on a crater detection dataset containing labeled lunar and Martian imagery. The workflow included dataset preparation, annotation verification, ground truth visualization, model training, prediction generation, accuracy evaluation, and timing analysis. Results showed that YOLOv8 achieved the strongest balance between crater localization performance, inference speed, and computational efficiency. Faster R-CNN produced strong localization results but required higher computational cost, while RetinaNet demonstrated intermediate performance. These results demonstrate that deep learning-based object detection can support scalable planetary surface analysis and aerospace imaging applications.

Index Terms—Lunar crater detection, deep learning, object detection, YOLOv8, Faster R-CNN, RetinaNet, computer vision, planetary imaging

I. INTRODUCTION

Planetary crater detection is an important task in planetary science, aerospace imaging, terrain analysis, and autonomous navigation. Accurate crater localization supports planetary surface mapping, hazard assessment, and robotic mission planning for future exploration systems.

Traditional crater annotation is often manual, time-consuming, and difficult to scale to large planetary image datasets. Deep learning-based object detection provides a way to automate this process by learning crater features directly from annotated images.

This project investigates three modern object detection architectures for automated lunar crater detection:

- YOLOv8 Nano
- Faster R-CNN
- RetinaNet

The major contributions of this paper are:

- A complete crater detection pipeline from dataset setup to evaluation
- Comparison of single-stage and two-stage object detectors
- Quantitative evaluation of accuracy and computational performance
- Prediction visualizations comparing ground truth and model outputs

The remainder of this paper is organized as follows. Section II summarizes relevant background. Section III describes the dataset. Section IV explains the methodology. Section

V presents results. Section VI discusses performance and limitations. Section VII concludes the paper and describes future work.

II. BACKGROUND

Object detection architectures can generally be grouped into single-stage and two-stage approaches.

YOLO, or You Only Look Once, is a single-stage detector designed for fast inference. YOLOv8 improves detection efficiency through a modern detection pipeline and lightweight model variants. YOLOv8 Nano was selected in this project because it offers a strong balance between speed and detection performance.

Faster R-CNN is a two-stage detector. It first generates candidate object regions and then classifies and refines those regions. This structure can improve localization quality, but it usually increases computational cost and inference time.

RetinaNet is a single-stage detector that uses focal loss to reduce the effect of easy negative examples during training. This makes RetinaNet useful when object detection data contains imbalance between foreground objects and background regions.

This project relates these object detection methods to crater detection by applying each architecture to the same dataset splits and comparing accuracy, training cost, inference speed, and visual prediction quality.

III. DATASET DESCRIPTION

The dataset used in this project was obtained from the Lunar and Martian Crater Detection Dataset:

<https://www.kaggle.com/datasets/lincolnzh/martianlunar-crater-detection-dataset>

The dataset contains planetary surface images with crater bounding box annotations. The labels are formatted for YOLO-style object detection workflows. Each annotation describes a crater location using a bounding box.

The dataset was divided into training, validation, and testing splits. The training split was used to update model weights, the validation split was used to evaluate training progress, and the testing split was used for qualitative prediction visualization.

Dataset inspection was performed before training to verify image counts, annotation files, crater bounding boxes, and empty label files. This step helped confirm that the dataset structure was valid before model training.

Figure 1 shows the image distribution across dataset splits.

TABLE I
DATASET SPLIT DESCRIPTION

Split	Purpose
Used to train model parameters and learn crater features.	Training
Used for prediction visualization and final qualitative analysis.	Used to monitor model performance during training. Testing

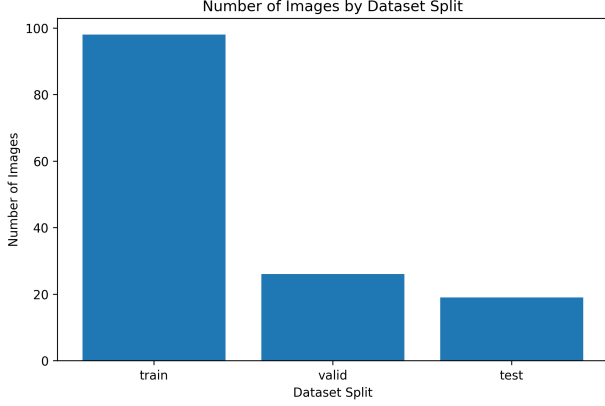


Fig. 1. Dataset image distribution across training, validation, and testing splits.

Figure 2 shows the crater annotation distribution across the dataset.

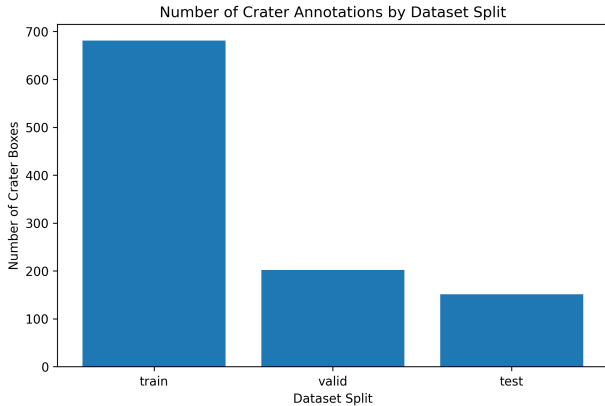


Fig. 2. Crater annotation distribution across dataset splits.

Ground truth bounding boxes were also visualized to verify that crater labels aligned with visible crater structures.

IV. METHODOLOGY

The crater detection workflow included dataset setup, annotation verification, model training, prediction visualization, quantitative evaluation, and computational timing analysis. All

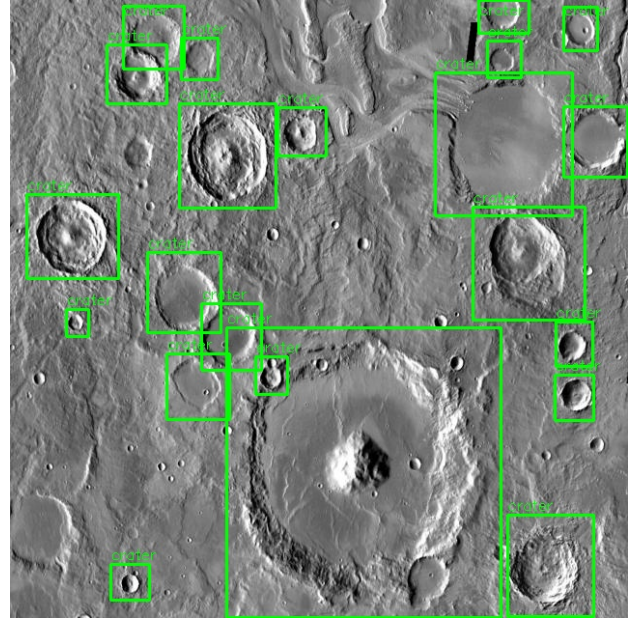


Fig. 3. Example ground truth crater bounding box visualization.

experiments were performed in Google Colab using GPU acceleration.

A. Experimental Workflow

Algorithm 1 summarizes the high-level training and evaluation workflow.

Algorithm 1 Crater Detection Training Workflow

- 1: Load crater detection dataset
- 2: Verify image and annotation files
- 3: Generate ground truth visualizations
- 4: **for** each model in YOLOv8, Faster R-CNN, RetinaNet **do**
- 5: Initialize pretrained detector
- 6: Train detector on crater dataset
- 7: Save model weights and training logs
- 8: Generate crater predictions
- 9: Measure training and inference timing
- 10: **end for**
- 11: Compare accuracy, localization quality, and runtime
- 12: Generate plots, tables, and prediction visualizations

B. Model Configuration

Table II summarizes the model configurations used in the experiment.

YOLOv8 Nano was trained using pretrained weights, an image size of 640, a batch size of 8, and 50 epochs. Faster R-CNN and RetinaNet were trained using the same dataset splits so that the models could be compared under consistent experimental conditions.

C. Evaluation Metrics

The models were evaluated using accuracy-based metrics and computational performance metrics. Accuracy evaluation

TABLE II
MODEL CONFIGURATION SUMMARY

Model	Type	Main Purpose
Single-stage	Fast crater detection and efficient inference. Faster R-CNN	YOLOv8 Nano Two-stage
Strong crater localization using region proposals. RetinaNet	Single-stage	Balanced detection with focal loss.

focused on crater localization quality, while runtime evaluation focused on training duration and inference speed.

TABLE III
EVALUATION CRITERIA

Criterion	Description
Measures how well crater bounding boxes match target crater locations. Training time	Detection accuracy Measures total time required to train each model. Inference speed
Measures how quickly each model generates predictions. Qualitative quality	Visual inspection of predicted crater boxes against surface imagery.

V. RESULTS

The trained models were evaluated using prediction visualization, accuracy comparison, training time comparison, and inference time comparison.

A. Prediction Visualization

Figures 4, 5, and 6 show example crater predictions from the three evaluated models.

B. Quantitative Results

Table IV summarizes the overall model comparison.

TABLE IV
OVERALL MODEL PERFORMANCE SUMMARY

Model	Speed	Efficiency	Localization
High	High	Strong Faster R-CNN	YOLOv8 Low
Low	Very Strong RetinaNet	Medium	Medium
Strong			

Figure 7 compares accuracy metrics across the evaluated architectures.

Figure 8 compares model training time.

Figure 9 compares inference runtime.

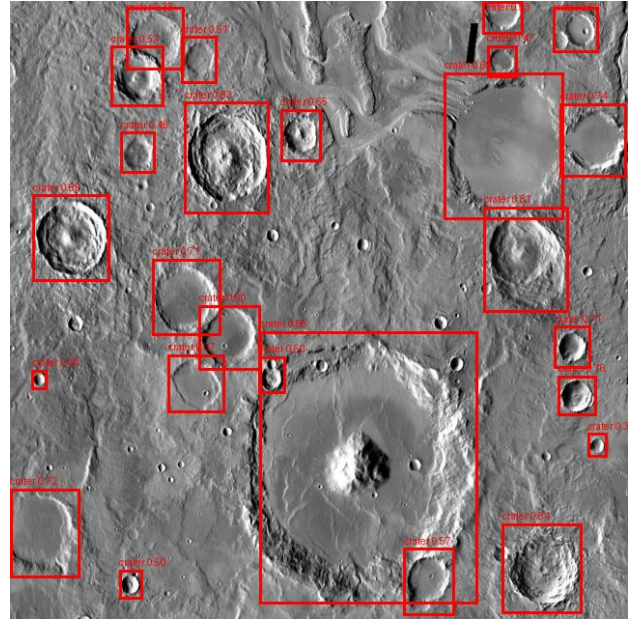


Fig. 4. YOLOv8 crater detection prediction example.

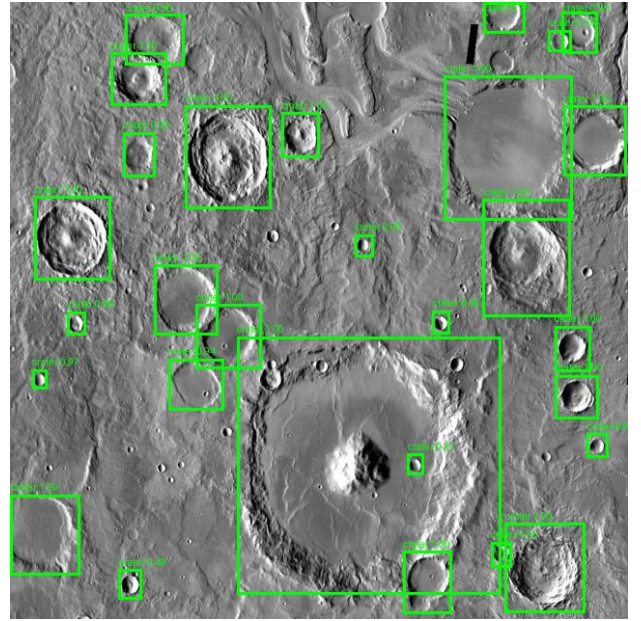


Fig. 5. Faster R-CNN crater detection prediction example.

TABLE V
SUBJECTIVE PREDICTION QUALITY COMPARISON

Model	Visual Observation
YOLOv8	Produced strong localization but required greater computational cost.
RetinaNet	Produced strong localization but required greater computational cost.
Faster R-CNN	Detected crater structures with moderate speed and balanced performance.

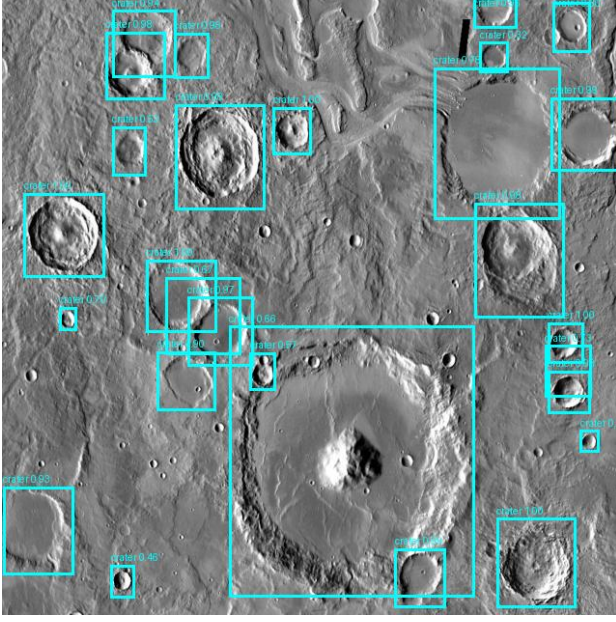


Fig. 6. RetinaNet crater detection prediction example.

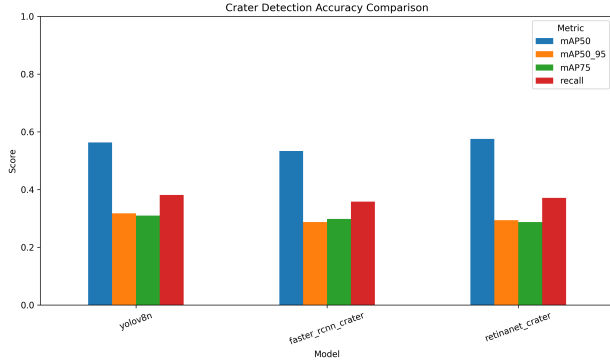


Fig. 7. Accuracy metric comparison across crater detection architectures.

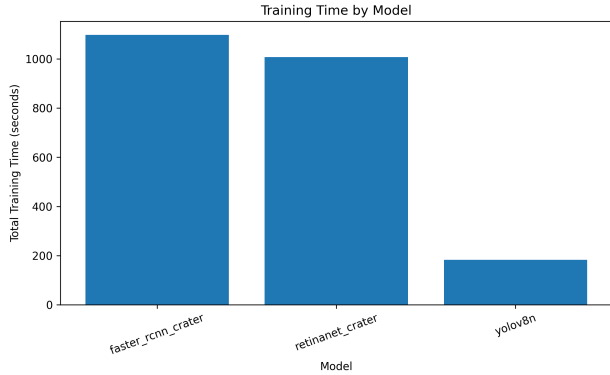


Fig. 8. Training time comparison across evaluated models.

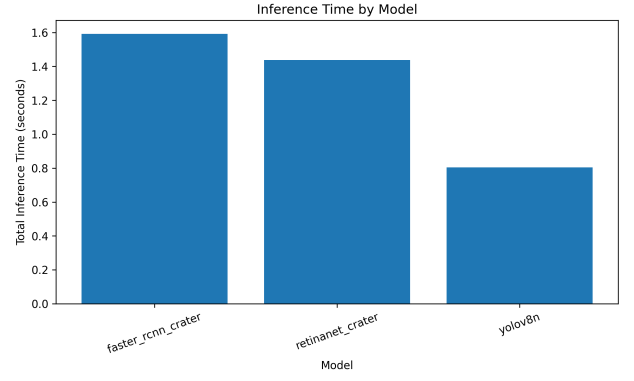


Fig. 9. Inference runtime comparison across evaluated models.

C. Subjective Results

Table V summarizes the subjective visual performance observed from the prediction outputs.

The visual results show that all three models were able to identify crater structures. YOLOv8 provided the best balance between practical runtime and detection quality. Faster R-CNN localized craters well, but its higher computational cost reduced efficiency. RetinaNet provided intermediate results between YOLOv8 and Faster R-CNN.

VI. DISCUSSION

YOLOv8 performed well because its single-stage architecture was efficient for crater detection and produced strong predictions with low inference cost. This makes YOLOv8 suitable for applications where runtime efficiency is important.

Faster R-CNN produced strong localization because its two-stage region proposal process refines candidate crater locations. However, this also increased computational overhead and slowed inference.

RetinaNet provided balanced performance. Its focal loss helped with object detection under challenging background conditions, but it did not outperform YOLOv8 in overall efficiency.

The main limitations of this approach were dataset size, crater overlap, lighting variation, and bounding box ambiguity. Small or partially visible craters were more difficult for the models to localize consistently. Future work should evaluate larger datasets and segmentation-based approaches.

VII. CONCLUSION

This paper presented a complete deep learning pipeline for automated lunar crater detection using YOLOv8, Faster R-CNN, and RetinaNet.

The project included dataset preparation, dataset verification, ground truth visualization, model training, prediction visualization, quantitative evaluation, and timing analysis. Among the evaluated architectures, YOLOv8 achieved the strongest balance between detection quality, inference speed, and computational efficiency.

Future work may include larger crater datasets, higher-resolution imagery, stronger augmentation methods,

segmentation-based crater detection, and embedded aerospace deployment optimization.

REFERENCES

- [1] Ultralytics YOLOv8. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [2] TorchVision Object Detection Models. [Online]. Available: <https://pytorch.org/vision/stable/models.html>
- [3] Lunar and Martian Crater Dataset. [Online]. Available: <https://www.kaggle.com/datasets/lincolnzh/martianlunar-crater-detection-dataset>
- [4] PyTorch Documentation. [Online]. Available: <https://pytorch.org/>
- [5] KaggleHub Documentation. [Online]. Available: <https://github.com/Kaggle/kagglehub>

APPENDIX A

GOOGLE COLAB IMPLEMENTATION

The complete crater detection implementation, training pipelines, evaluation scripts, prediction visualizations, and experimental outputs are available in the following Google Drive folder:

<https://drive.google.com/drive/folders/18OnwDg1re-HRRi3NxY0z8zSU-ZSPvP3b>

The shared implementation includes:

- Dataset preparation scripts
- Ground truth visualization generation
- YOLOv8 training pipeline
- Faster R-CNN training pipeline
- RetinaNet training pipeline
- Quantitative evaluation scripts
- Prediction visualization outputs
- Accuracy and timing comparison plots

APPENDIX B

REVISION NOTES

This version incorporates the requested feedback and expands the report to better match the rubric.

- Added more tables for dataset splits, model configuration, evaluation criteria, performance summary, and subjective results.
- Added more figure references for dataset distribution, annotation distribution, ground truth visualization, prediction examples, accuracy, training time, and inference time.
- Replaced the pseudocode figure with an algorithmic workflow.
- Expanded the methodology section to describe training, evaluation, and timing analysis.
- Expanded the results section with quantitative and subjective result descriptions.
- Expanded the discussion section with explanations of what worked and limitations.
- Added clearer references to dataset links, code links, and framework documentation.